and middleware on a virtual machine to make your application run. Instead, a container can run reliably in almost any computing environment, independent of the application, its dependencies, or middleware. It doesn't matter which kind of application it is, in which language it's written, or which libraries it uses. All containers have a similar shape from the outside, just like the containers used for shipping.

Therefore, containers enable agility, portability across different environments, and deployment repeatability. Being lightweight and less resource-demanding, they are perfect for running in the cloud, where applications are disposable and scaled dynamically and quickly. In comparison, building and destroying virtual machines is much more expensive and time-consuming.

Containers! Containers everywhere!

Container is one of those words that can mean different things. Sometimes this ambiguity can generate confusion, so let's see what it means in specific contexts.

- *OS*—An OS container is a method for running one or more processes in an environment isolated from the rest of the system. This book will focus on *Linux containers*, but note that Windows containers also exist.
- *Docker*—A Docker container is an implementation of a Linux container.
- *OCI*—An OCI container is a standardization of the Docker container implementation by the Open Container Initiative (OCI).
- *Spring*—A Spring container is the application context where objects, properties, and other application resources are managed and executed.
- *Servlet*—A Servlet container provides a runtime for web applications leveraging the Java Servlet API. The Catalina component from the Tomcat server is an example of a Servlet container.